OO Programozás

Operátor felültöltés, másolás és átalakítás

Dr. Hatwagner F. Miklós

Széchenyi István Egyetem, Győr

https://github.com/wajzy/GKxB_INTM119

2026. november 5.





Jelenlegi legjobb tudásunk szerint készítsünk osztályt egy komplex szám tárolására és műveletek elvégzésére, azaz:

- tároljuk a szám valós és képzetes részét,
- készítsünk konstruktorokat,
- getter/setter tagfüggvényeket,
- és olyan függvényeket, melyekkel komplex számok összeadhatók és szorozhatók!

Complex1.hpp

```
 6  class Complex {
 7      double re, im;
 8    public:
 9      Complex() {
10        re = im = 0.;
11      }
12      Complex(double re, double im) {
13        this->re = re;
14        this->im = im;
15      }
16      double getRe() const {
17        return re;
18      }
19      void setRe(double re) {
20        this->re = re;
21      }
```

Complex1.hpp

```
22      double getIm() const {
23        return im;
24      }
25      void setIm(double im) {
26        this->im = im;
27      }
28      void print() const {
29        std::cout << re << '+' << im << 'i' << std::endl;
30      }
31      Complex add(Complex right) const {
32        return Complex(re + right.re, im + right.im);
33      }
34      Complex mult(Complex right) const {
35        return Complex(re*right.re - im*right.im, im*right.re + re*right.im);
36      }
37  };
```

main1.cpp

```
1  #include <iostream>
2  #include "Complex1.hpp"
3
4  int main() {
5    Complex c(1., 1.);
6    Complex sum = c.add(Complex(2., 2.));
7    sum.print(); // 3+3i
8    Complex total = c.mult(Complex(3., 4.));
9    total.print(); // -1+7i
10 }
```

Operátor felültöltés

- A C++ megengedi, hogy az operátorok jelentését kiterjesszük a saját típusainkra (osztályainkra)
- Pl. ha értelmezhető az összeadás két `int` vagy `float` között, akkor két `Complex` objektum miért ne lehetne összeadható?
- Az operátorok működését (praktikusan nyilvános tag)függvények adják meg → `operatorX`, ahol `X` pl. `+`, `*`.

Complex2.hpp

```
31      // Use reference to avoid copy of 'right'
32      Complex operator+(const Complex& right) const {
33        return Complex(re + right.re, im + right.im);
34      }
35      Complex operator*(const Complex& right) const {
36        return Complex(re*right.re - im*right.im, im*right.re + re*right.im);
37      }
```

- + művelet bal operandusa: aktuális objektum, a jobb oldalit paraméterként kapja.
- Utóbbi felesleges (tagonkénti) másolásának elkerülésére referenciát használunk.
- Új, ideiglenes (eredmény) objektum jön létre e kettő alapján, ezt adja vissza.

main2.cpp

```
4  int main() {
5    Complex c(1., 1.);
6    // Complex sum = c.operator+(Complex(2., 2.));
7    Complex sum = c + Complex(2., 2.);
8    sum.print();
9    // Complex sum2 = c + 100.;
10   // error: no match for 'operator+'
11   // (operand types are 'Complex' and 'double')
12   // note:  no known conversion for argument 1
13   // from 'double' to 'const Complex&'
14   Complex total = c * Complex(3., 4.);
15   total.print();
16 }
```

Probléma:

Összeadásnál a jobb oldali operandusnak `Complex`-nek kell lennie.

Megoldás:

További felültöltött operátor függvények hozzáadása, pl. `double`-t hozzáadhatunk a valós részhez.

Complex3.hpp

```
35      Complex operator+(double re) const {
36        return Complex(this->re + re, im);
37      }
```

main3.cpp

```
4  int main() {
5    Complex c(1., 1.);
6    // Complex sum = c.operator+(Complex(2., 2.));
7    Complex sum = c + Complex(2., 2.);
8    sum.print();
9    // Complex sum2 = c.operator+(100.);
10   Complex sum2 = c + 100.;
11   sum2.print();
12   // Complex sum3 = 100. + c;
13   // error: no match for 'operator+'
14   // (operand types are 'double' and 'Complex')
15   Complex total = c * Complex(3., 4.);
16   total.print();
17 }
```

- Ez sem segít, ha a *bal* operandus `double` típusú → *nem tag*, két paraméteres `operator` függvény.
- Nem éri el a privát/védett tagokat → barát (`friend`) függvény: mindenhez hozzáfér az osztályon belül.
- Másik osztály is lehet az osztályunk barátja, pl.:
  `friend class FriendOfComplex;`

Complex4.hpp

```
 6  class Complex {
 7      double re, im;
 8    public:

28      friend std::ostream& operator<<(std::ostream& os, const Complex& cplx);
29
30      Complex operator+(const Complex& right) const {
31        return Complex(re + right.re, im + right.im);
32      }
33      Complex operator+(double re) const {
34        return Complex(this->re + re, im);
35      }
36      friend Complex operator+(double re, const Complex& right);
```

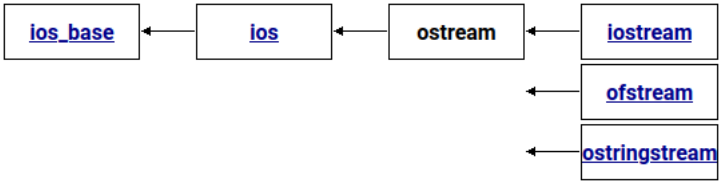
Complex4.cpp

```
3  std::ostream& operator<<(std::ostream& os, const Complex& c) {
4    return os << c.re << '+' << c.im << 'i';
5  }
6
7  Complex operator+(double re, const Complex& right) {
8    return Complex(re + right.re, right.im);
9  }
```

main4.cpp

```
4  int main() {
5    Complex c(1., 1.);
6    // Complex sum = c.operator+(Complex(2., 2.));
7    Complex sum = c + Complex(2., 2.);
8    std::cout << sum << std::endl; // 3+3i
9    // Complex sum2 = c.operator+(100.);
10   Complex sum2 = c + 100.;
11   std::cout << sum2 << std::endl; // 101+1i
12   // Complex sum3 = operator+(100., c);
13   Complex sum3 = 100. + c;
14   std::cout << sum3 << std::endl; // 101+1i
15   Complex total = c * Complex(3., 4.);
16   std::cout << total << std::endl; // -1+7i
17 }
```

Az `ostream` a `cout`, `cerr` és `clog` objektumok típusa.



Ha `operator<<` tagfüggvény lenne, akkor az `std::ostream` operátoraként kellene megvalósítani, de ezt nem tehetjük.

Valójában csupa nem tag barát függvénnyel is megoldhattuk volna az operátorok felültöltését, sőt, a kétoperandusú tagfüggvényeket ajánlott (Item 24) is így elkészíteni
→ konverziós konstruktor, automatikus típuskonverzió

Complex5.hpp (Complex5.cpp)

```
 6  class Complex {
 7      double re, im;
 8    public:

28      friend std::ostream& operator<<(std::ostream& os, const Complex& cplx);
29
30      friend Complex operator+(const Complex& left, const Complex& right);
31      friend Complex operator+(const Complex& left, double re);
32      friend Complex operator+(double re, const Complex& right);
33
34      friend Complex operator*(const Complex& left, const Complex& right);
35      friend Complex operator*(const Complex& left, double re);
36      friend Complex operator*(double re, const Complex& right);
37  };
```

Készítsünk időt reprezentáló osztályt, ami 24 órás formátumban, óra, perc, mp. tagokkal tárolja az időt! Egyebek mellett töltsük felül a növelő és csökkentő operátorokat!

Time.hpp

```
6  class Time {
7      int hour;
8      int minute;
9      int second;
10
11     // Normalizes time (60sec = 1min, etc.)
12     void normalize() {
13         if(second >= 60) {
14             minute += second / 60;
15             second %= 60;
16         } else if(second < 0) {
17             int borrow = (-second + 59) / 60;
18             minute -= borrow;
19             second += borrow * 60;
20         }
```

Time.hpp

```
22          if(minute >= 60) {
23              hour += minute / 60;
24              minute %= 60;
25          } else if(minute < 0) {
26              int borrow = (-minute + 59) / 60;
27              hour -= borrow;
28              minute += borrow * 60;
29          }
30
31          // 24 hour circulation
32          hour = (hour % 24 + 24) % 24;
33      }
```

Az átvitelek kezelésénél figyelembe vettük, hogy a % b eredménye mindig a előjelét kapja, azaz pl. -1 % 24 == -1.

Time.hpp

```
35 public:
36     Time(int h = 0, int m = 0, int s = 0) : hour(h), minute(m), second(s) {
37         normalize();
38     }
39
40     // Prefix ++
41     Time& operator++() {
42         ++second;
43         normalize();
44         return *this;
45     }
46
47     // Postfix ++
48     Time operator++(int) {
49         Time temp = *this;
50         ++(*this);
51         return temp;
52     }
```

Fontos észrevenni:

- A *prefix* változat referenciát ad vissza, a *postfix* viszont értéket, másolatot szolgáltat a korábbi állapotról (és nem adhatja lokális változó referenciáját)
- A *postfix* alakot az `int` „paraméter” különbözteti meg a *prefix* változattól.
- A *prefix* változat gyorsabb, mert nem kell másolatot készítenie.
- A csökkentés operátora analóg módon készíthető el.

Time.hpp

```
68      // Inline friend operator
69      friend std::ostream& operator<<(std::ostream& os, const Time& t) {
70          return os << std::setfill('0')
71             << std::setw(2) << t.hour << ":"
72             << std::setw(2) << t.minute << ":"
73             << std::setw(2) << t.second;
74      }
75  };
```

A barát függvény definíciója most az osztály blokkján belülre került, de ennek egyetlen hatása az, hogy a függvény implicit módon `inline`-ná válik. Az időegységek két karakter szélesen, elölnullázva jelennek meg. (Ld. iomanip fejfájl.)

TimeMain.cpp

```
4  int main() {
5      Time t1(23, 59, 58);
6      std::cout << "Initial time: " << t1 << std::endl;
7
8      std::cout << "++t1 -> " << ++t1 << std::endl;
9      std::cout << "t1++ -> " << t1++ << " (return value)\n";
10     std::cout << "after increasing t: " << t1 << std::endl;
11
12     std::cout << "--t1 -> " << --t1 << std::endl;
13     std::cout << "t1-- -> " << t1-- << " (return value)\n";
14     std::cout << "after reducing t: " << t1 << std::endl;
15
16     Time t2(0, 0, -5);
17     std::cout << "Initial time: " << t2 << std::endl;
18 }
```

Kimenet

```
Inital time: 23:59:58
++t1 -> 23:59:59
t1++ -> 23:59:59 (return value)
after increasing t: 00:00:00
--t1 -> 23:59:59
t1-- -> 23:59:59 (return value)
after reducing t: 23:59:58
Inital time: 23:59:55
```

Operátor felültöltés

- Definiáltuk az operátor *jelentését* olyan kifejezésben, melyben legalább egy operandus az osztály objektuma.
- Nem változtatható meg az operátor *szintakszisa*, *precedenciája*, *asszociativitása*, *operandusainak száma*.
- *Majdnem* minden operátor felültölthető (kivétel pl. ::, .).

- Az értékadás megengedett objektumok között → tagonkénti másolás.
- Probléma: dinamikusan foglalt területet több objektum adattagja is címezhet. Ha az egyik destruktora ezt felszabadítja, a másik ezen próbálhat műveletet végezni vagy újra felszabadítani.
- Térjünk vissza a Message osztályhoz, lássuk el felültöltött értékadás operátorral, készítsünk másolatot a tárolt szövegről!

message2.cpp

```
4  class Message {
5    private:
6      char* pStr;
7    public:
8      Message() {
9        pStr = new char[1]{'\0'};
10       std::cout << "Created␣[" << this << "]\n";
11     }
12
13     Message(const char* s) {
14       pStr = new char[strlen(s) + 1];
15       strcpy(pStr, s);
16       std::cout << "Created␣[" << this << ",␣" << pStr << "]\n";
17     }
```

message2.cpp

```
19      ~Message() {
20        std::cout << "Freed [" << this << ", " << pStr << "]\n";
21        delete[] pStr;
22      }
23
24      friend std::ostream& operator<<(std::ostream& os, const Message& m);
25
26      // return reference instead of void to allow multiple assignments
27      Message& operator=(const Message& m) {
28        // do not copy to itself
29        if(&m == this) return *this;
30        setMessage(m.pStr);
31        return *this;
32      }
```

message2.cpp

```
34      void setMessage(const char* s) {
35        delete[] pStr;
36        pStr = new char[strlen(s) + 1];
37        strcpy(pStr, s);
38      }
39  };
40
41  std::ostream& operator<<(std::ostream& os, const Message& m) {
42    return os << m.pStr;
43  }
```

message2.cpp

```
45 int main() {
46   Message m1;
47   m1.setMessage("Hello C++ world!");
48   std::cout << m1 << std::endl;
49
50   Message m2;
51   m2 = m1;
52   std::cout << m2 << std::endl;
53
54   Message m3;
55   m3 = m2 = m1;
56   std::cout << m3 << std::endl;
57
58   m1 = m1;
59   std::cout << m1 << std::endl;
60
61   Message m4 = m1; // double free!
62   std::cout << m4 << std::endl;
63 }
```

Kimenet

```
Created [0x7ffcf5018618]
Hello C++ world!
Created [0x7ffcf5018620]
Hello C++ world!
Created [0x7ffcf5018628]
Hello C++ world!
Hello C++ world!
Hello C++ world!
Freed [0x7ffcf5018630, Hello C++ world!]
Freed [0x7ffcf5018628, Hello C++ world!]
Freed [0x7ffcf5018620, Hello C++ world!]
Freed [0x7ffcf5018618, ]
free(): double free detected in tcache 2
Aborted (core dumped)
```

Ami jól működik:

Az értékadás operátor, ami másolatot hoz létre a karakterláncból,
nem engedi magát önmagára másolni, és
a többszörös értékadás is működik a visszaadott referencia miatt.

Ami hibás:

Az inicializáció nem értékadás, ilyenkor nem fut le a felültöltött függvényünk!

Másolási konstruktor (copy constructor):

Akár *egyetlen, azonos típusra vonatkozó referencia* paraméterrel is hívható konstruktor. (Lehetnek további paraméterek alapértelmezett értékekkel.)

Konverziós konstruktor (conversion constructor):

Ez is hívható egyetlen paraméterrel, de az más típusú.

Közös tulajdonság: objektum *inicializálásának* lehetősége kétféle szintaktikával:

`Osztaly obj(42);` $\equiv$ `Osztaly obj = 42;`

Másolási konstruktor további tulajdonságai:

- Ha mi nem készítünk, a fordító létrehoz egyet → tagonkénti másolás.
- Impliciten hívásra kerül:
  - Objektum átadása függvénynek paraméterként → a végtelen rekurzió elkerülése miatt kell az első paraméternek referenciának lennie.
  - Függvény adott osztály objektumát adja vissza.

message3.cpp

```
13      Message(const char* s) { // conversion ctor
14        pStr = new char[strlen(s) + 1];
15        strcpy(pStr, s);
16        std::cout << "Conversion ctor [" << this << "," << pStr << "]\n";
17      }
18
19      /* for old compilers
20      Message(const Message& m) {
21        pStr = new char[strlen(m.pStr) + 1];
22        strcpy(pStr, m.pStr);
23        std::cout << "Created [" << this << ", " << m.pStr << "]\n";
24      } */
25
26      // C++11+ / delegating constructors
27      Message(const Message& m) : Message(m.pStr) {
28        std::cout << "(Copy ctor -> conversion ctor)\n";
29      }
```

message3.cpp

```
58 int main() {
59   Message m1;
60   m1.setMessage("Hello C++ world!");
61   std::cout << m1 << std::endl;

74   Message m4 = m1; // calls copy ctor
75   std::cout << m4 << std::endl;
76   Message m5(m1); // calls copy ctor
77   std::cout << m5 << std::endl;
78   Message m6 = "m6";
79   Message m7("m7");
80 }
```

Kimenet

```
Default ctor [0x7ffed72bf210]
Hello C++ world!
...
Conversion ctor [0x7ffed72bf228, Hello C++ world!]
(Copy ctor --> conversion ctor)
Hello C++ world!
Conversion ctor [0x7ffed72bf230, Hello C++ world!]
(Copy ctor --> conversion ctor)
Hello C++ world!
Conversion ctor [0x7ffed72bf238, m6]
Conversion ctor [0x7ffed72bf240, m7]
Freed [0x7ffed72bf240, m7]
Freed [0x7ffed72bf238, m6]
Freed [0x7ffed72bf230, Hello C++ world!]
...
Freed [0x7ffed72bf210, Hello C++ world!]
```

Mikor hívjuk a konverziós konstruktort?

- Explicit hívás, más típusú adattal.
- Implicit hívás, a fordító egy adatot az osztály típusára akar alakítani $\rightarrow$ lehetővé teszi a felültöltött operátorfüggvények számának csökkentését!

Complex6.hpp

```
 6  class Complex {
 7      double re, im;
 8    public:
 9      // explicitely defaulted constructor
10      Complex() = default;
11      // conversion constructor
12      Complex(double re, double im = 0.) {
13        this->re = re;
14        this->im = im;
15      }

29      friend Complex operator+(const Complex& left, const Complex& right);
30      friend Complex operator*(const Complex& left, const Complex& right);
31  };
```

Complex6.hpp

```
10      Complex() = default;
```

Ha készítünk legalább egy konstruktort, a fordító nem hoz létre automatikusan *alapértelmezett* konstruktort. Ha mégis szükség lenne rá: `default` (C++11+; használható a kulcsszó más *speciális* tagfüggvényekhez is.)

Legtöbbször ha szükség van az alábbiak közül akár csak egynek is a definiálására, akkor a másik kettőre is szükség van (Rule of Three):

- destruktor
- másoló konstruktor
- hozzárendelés operátor

Vagy, átmeneti jelleggel akár le is tilthatjuk egyes függvények alapértelmezett előállítását → `delete`
Hasznos lehet nem kívánt típuskonverziók tiltására is.

message4.cpp

```
4  class Message {

7    public:

19     // C++11: deleted constructor
20     Message(const Message& m) = delete;

29     // deleted assignment operator
30     Message& operator=(const Message& m) = delete;

37 };
```

message4.cpp

```
44  int main() {

49    Message m2;
50    // m2 = m1; error: use of deleted function
51    // 'Message& Message::operator=(const Message&)'
52
53    // Message m4 = m1; error: use of deleted function
54    // 'Message::Message(const Message&)'
55
56    // Message m6 = "Conversion + copy ctor"; error: use of deleted function
57    // 'Message::Message(const Message&)'
58    // after user-defined conversion: Message::Message(const char*)
59
60    Message m7("Conversion ctor"); // OK
61  }
```

Az `std::cin` folyam működése során különféle hibák fordulhatnak elő, melyek hibajelző biteket állítanak be. Pl. ha egyetlen, bemenetről érkező karaktert sem tud feldolgozni, akkor magasra vált a `fail` bit.

bool1.cpp

```
3  int main() {
4    std::cout << "Please␣enter␣a␣number:␣";
5    int number;
6    std::cin >> number;
7    if(std::cin.fail()) {
8      std::cout << "Something␣went␣wrong.\n";
9    } else {
10     std::cout << "The␣entered␣number␣is:␣" << number << std::endl;
11   }
12 }
```

Helyes adatok

```
Please enter a number: 123
The entered number is: 123
```

Hibás adatok

```
Please enter a number: horse
Something went wrong.
```

Ugyanez rövidebben, felhasználva, hogy az `std::cin` objektum képes automatikusan logikai értékké alakulni feltételes kifejezésekben:

bool2.cpp

```
5   int number;
6   std::cin >> number;
7   if(!std::cin) {
8     std::cout << "Something went wrong.\n";
9   } else {
10    std::cout << "The entered number is: " << number << std::endl;
11  }
```

Sőt, felhasználva, hogy a >> operátor az objektum referenciájával tér vissza:

bool3.cpp

```
5   int number;
6   if(std::cin >> number) {
7     std::cout << "The entered number is: " << number << std::endl;
8   } else {
9     std::cout << "Something went wrong.\n";
10  }
```

Fejlesszük tovább a Message osztályt, hogy üres karakterlánc esetén logikai hamisat, egyébként igazat adjon!

message5.cpp

```
4  class Message {
5    private:
6      char* pStr;
7    public:

25     operator bool() const {
26         return pStr != nullptr && pStr[0] != '\0';
27     }

40 };
```

message5.cpp

```
46 int main() {
47   Message m1 = "Important message.";
48   if(m1) {
49     std::cout << "The 1st message is: " << m1 << '\n';
50   } else {
51     std::cout << "No 1st message.\n";
52   }
53
54   Message m2;
55   if(m2) {
56     std::cout << "The 2nd message is: " << m2 << '\n';
57   } else {
58     std::cout << "No 2nd message.\n";
59   }
60
61   int meaningless = m1 + m2;
62   std::cout << "meaningless = " << meaningless << '\n';
63 }
```

Kimenet

```
The 1st message is: Important message.
No 2nd message.
meaningless = 1
```

Az operátor (és a `bool` típusra alakítás) működik ugyan, de az adott probléma szempontjából értelmetlen átalakítások is végbemennek → utóbbiak tiltása az `explicit` kulcsszóval

message6.cpp

```
4   class Message {
5     private:
6       char* pStr;
7     public:


25      explicit operator bool() const {
26          return pStr != nullptr && pStr[0] != '\0';
27      }


40  };


46  int main() {


61    // error: no match for 'operator+' (operand types are 'Message' and 'Message')
62    // int meaningless = m1 + m2;
63    // std::cout << "meaningless = " << meaningless << '\n';
64  }
```

Minden, *egy paraméterrel hívható konstruktort* is tegyünk `explicit`-té, hacsak nincs nyomós okunk az ellenkezőjére (C++ Core Guidelines, C.46)!

Szempont	Implicit megengedett	`explicit` kötelező / ajánlott
Szemantikai kapcsolat	„Ugyanaz az érték” (érték-egyenértékűség), pl. `double` → `Complex`, `const char*` → `Message`	„Csak egy paraméter/tulajdonság”, pl. `int` (méret) → `IntArray`
Költség	Olcsó, közvetlen átalakítás	Drága memóriaallokáció, erőforrás-foglalás (pl. fájl, mutex)
Cél	Matematikai szimmetria / kommutativitás (operátorok működése: 2 + c)	Véletlen, meglepő ideiglenes objektumok megakadályozása

Néhány példa, amikor érdemes tiltani az implicit konverziót

A paraméter nem egyenértékű érték, csak egy mellékes tulajdonság

```
class IntArray {
public:
    IntArray(int size);
};

void printArray(const IntArray& arr);

int main() {
    printArray(10); // meaningless, but compiles
}
```

Drága erőforrás használatával járó művelet

```
class File {
public:
    File(const char* path);
};

void logToFile(const File& f);

int main() {
    logToFile("config.log");
    // opens a file, passes it, and then closes it
    // after logging a single line!
}
```

Nem szándékolt függvény-túlterhelési (overload) kétértelműségek

```
class BigInt {
public:
    BigInt(int value);
};

class Fraction {
public:
    Fraction(int num);
};

void process(const BigInt& b);
void process(const Fraction& f);

int main() {
    process(42); // call of overloaded 'process(int)' is ambiguous!
}
```